Programação Orientada a Objetos

Departamento de Computação
Universidade Federal de Sergipe

Framework Collections

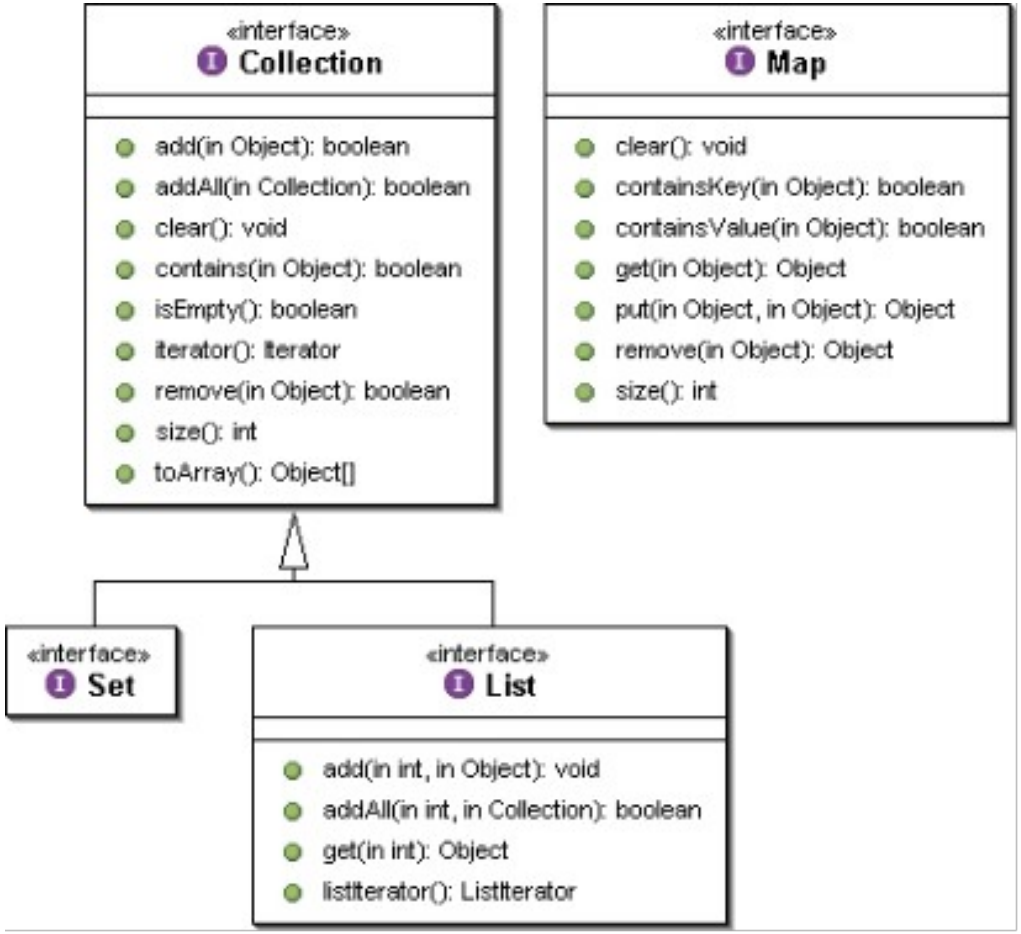
Prof. Kalil Araujo Bispo
kalil@dcomp.ufs.br

Introdução

- **Problemas com o uso de arrays**
  - Não é possível redimensioná-los
  - Não há busca direta por um elemento com índice desconhecido
  - Não é possível saber posições usadas sem criar métodos auxiliares

- **Conjunto de classes que implementam estruturas de dados básicas e avançadas**
  - Framework Collections, do java.util

Coleções


«interface»
Collection
add(in Object): boolean
addAll(in Collection): boolean
clear(): void
contains(in Object): boolean
isEmpty(): boolean
iterator(): Iterator
remove(in Object): boolean
size(): int
toArray(): Object[]
«interface»
Map
clear(): void
containsKey(in Object): boolean
containsValue(in Object): boolean
get(in Object): Object
put(in Object, in Object): Object
remove(in Object): Object
size(): int
«interface»
Set
«interface»
List
add(in int, in Object): void
addAll(in int, in Collection): boolean
get(in int): Object
listIterator(): ListIterator

Interfave java.util.Collection

- **Base para as coleções**
- **Definem métodos como**

`boolean add(Object)`	Adiciona um elemento na coleção. Como algumas coleções não suportam elementos duplicados, este método retorna true ou false indicando se a adição foi efetuada com sucesso.
`boolean remove(Object)`	Remove determinado elemento da coleção. Se ele não existia, retorna false.
`int size()`	Retorna a quantidade de elementos existentes na coleção.
`boolean contains(Object)`	Procura por determinado elemento na coleção, e retorna verdadeiro caso ele exista. Esta comparação é feita baseando-se no método equals() do objeto, e não através do operador ==.
`Iterator iterator()`	Retorna um objeto que possibilita percorrer os elementos daquela coleção.

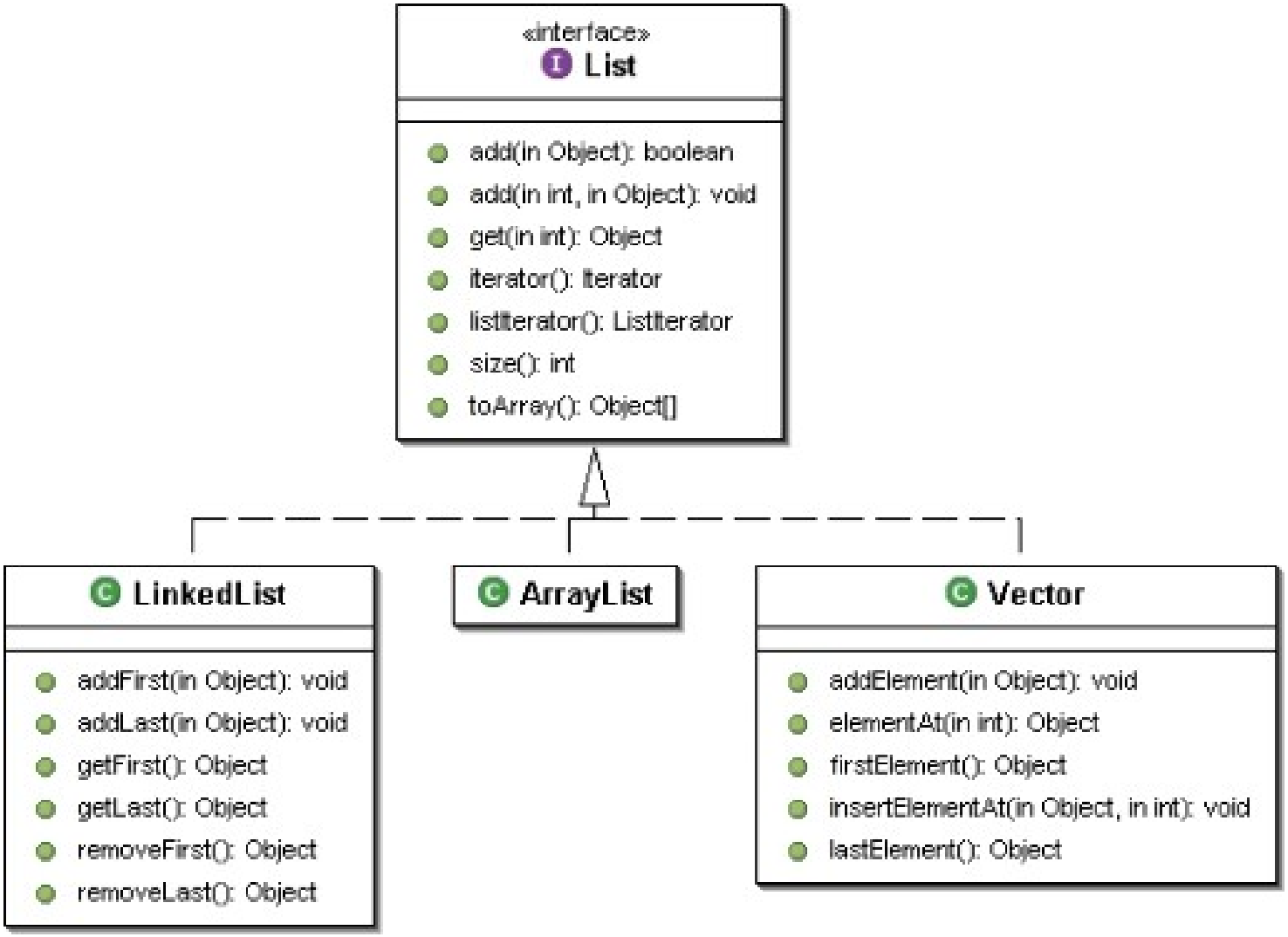
Coleções

- **Uma coleção pode implementar diretamente a interface Collection**
- **Normalmente se usa uma das duas subinterfaces mais conhecidas: Set e List**
- **Set**
  - Não permite repetição de elementos
  - Busca por elementos pode ser mais rápida por causa do uso de hash tables
- **List**
  - Permite elementos duplicados
  - Mantém ordenação
- **Queue**
  - Inserção e remoção com critério (LIFO, FIFO, prioridades)

List (java.util.List)

- **Coleção que permite elementos duplicados e mantém uma ordenação específica entre os elementos**
- **Interface List especifica o que uma classe deve ser capaz de fazer para ser uma lista**
  - Remover, adicionar, obter tamanho, buscar elementos...

List (java.util.List)



List (java.util.List) - Implementações

- **A interface List possui dois métodos add**
  - Um que recebe o objeto a ser inserido e o coloca no final da lista
  - Um que permite adicionar o elemento em qualquer posição da mesma
- **Todos os métodos trabalham com Object**

List (java.util.List) - Implementações

- **ArrayList**
  - Uma das mais utilizadas
  - Trabalha com um array interno para gerar uma lista
  - Mais rápida no acesso aleatório
  - Não se acessa o array diretamente, há métodos que encapsulam esse array

```
List lista = new ArrayList();
lista.add("Manoel");
lista.add("Joaquim");
lista.add("Maria");
```

List (java.util.List) - Implementações

```
ContaCorrente c1 = new ContaCorrente();
c1.deposita(100);

ContaCorrente c2 = new ContaCorrente();
c2.deposita(200);

ContaCorrente c3 = new ContaCorrente();
c3.deposita(300);

List contas = new ArrayList();
contas.add(c1);
contas.add(c3);
contas.add(c2);

System.out.println(contas.size());
```

List (java.util.List) - Implementações

```
for (int i = 0; i < contas.size(); i++) {
   contas.get(i);
   ...
}
```

```
for (int i = 0; i < contas.size(); i++) {
   ContaCorrente cc = (ContaCorrente) contas.get(i);
   System.out.println(cc.getSaldo());
}
```

List (java.util.List) - Implementações

- **LinkedList**
  - Lista encadeada: métodos de remoção e inserção no primeiro e último elemento da lista
  - Mais rápida na inserção e remoção de itens nas pontas
- **Vector**
  - Lida de maneira diferente com processos correndo em paralelo
  - Possui custo adicional em relação a ArrayList quando não há acesso simultâneo aos dados

List (java.util.List)

- **Em qualquer lista é possível colocar qualquer Object**
  - Pode-se misturar objetos
  - Para restringir a lista a objetos de uma classe específica, é usado o recurso Generics

Generics

- Restringe as coleções a um determinado tipo de objetos
- Causa erro de compilação quando objetos de classes diferentes forem utilizados como parâmetro
- Elimina a necessidade de casting

Generics

```
List<ContaCorrente> contas = new
ArrayList<ContaCorrente>();

contas.add(c1);
contas.add(c3);
contas.add(c2);

for(int i = 0; i < contas.size(); i++) {
   ContaCorrente cc = contas.get(i);
   System.out.println(cc.getSaldo());
}
```

Generics

- **A partir do Java 7, se um tipo genérico for instanciado na mesma linha de sua declaração, não é necessário passar os tipos novamente**
  - Operador diamante

```
List<ContaCorrente> contas = new ArrayList<>();

contas.add(c1);
contas.add(c3);
contas.add(c2);

for(int i = 0; i < contas.size(); i++) {
   ContaCorrente cc = contas.get(i);
   System.out.println(cc.getSaldo());
}
```

Uso das Interfaces

- **É importante sempre usar referências à interface da coleção, e não às suas implementações**
  - O ideal é sempre trabalhar com a interface mais genérica possível
  - Evita comprometimento com implementação específica
  - Baixo acoplamento: pode-se trocar a implementação facilmente, já que a programação é voltada aos métodos da interface

Ordenação

- Método estático sort, da classe Collections
- Recebe um List como parâmetro

```
List<String> lista = new ArrayList<>();

lista.add("Sérgio");
lista.add("Paulo");
lista.add("Guilherme");

//toString de ArrayList é redefinido:
System.out.println(lista);
Collections.sort(lista);
System.out.println(lista);
```

Ordenação

- **Ordenação de listas de objetos de classes definidas pelo programador**
- **Deve-se definir um critério de ordenação, através da implementação da interface java.lang.Comparable**
  - Define método int compareTo(Object)
- **Retorno:**
  - Zero se objetos forem iguais
  - Valor negativo se o objeto for menor que o objeto recebido como parâmetro
  - Valor positivo se o objeto for maior que o objeto recebido como parâmetro

Ordenação

```
public class ContaCorrente extends Conta
implements Comparable<ContaCorrente> {

...

   public int compareTo(ContaCorrente outra) {
      if (this.saldo < outra.saldo) {
         return -1;
      }
      if (this.saldo > outra.saldo) {
         return 1;
      }
      return 0;
   }
}
```

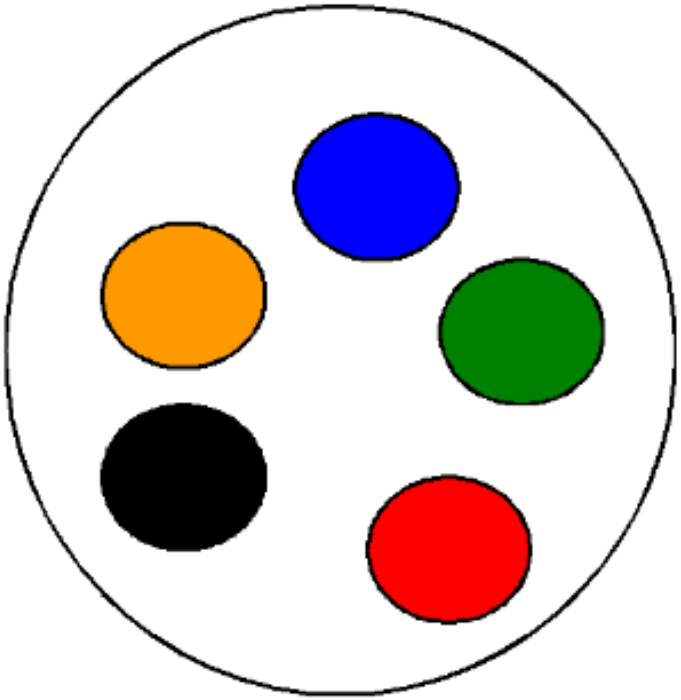
Ordenação

- **É possível definir outros critérios de ordenação usando um objeto do tipo Comparator**
- **Há um método sort em Collections que recebe, além da List, um Comparator definindo um critério de ordenação específico**
- **A classe Collections traz outros métodos estáticos, como:**
  - binarysearch(List, Object)
  - max(Collection)
  - min(Collection)
  - reverse(List)
  - ...

Set (java.util.Set)

- **Interface que representa uma coleção que não permite elementos duplicados**
- **A ordem em que os elementos são armazenados pode não ser a ordem na qual eles foram inseridos no conjunto**
  - Comportamento varia de implementação para implementação

Set



Possíveis ações em um conjunto:

- A camiseta Azul está no conjunto?
- Remova a camiseta Azul.
- Adicione a camiseta Vermelha.
- Limpe o conjunto.

- Não existem elementos duplicados!

- Ao percorrer um conjunto, sua ordem não é conhecida!

Set

- **Principais implementações:**
  - Classes HashSet, LinkedHashSet e TreeSet

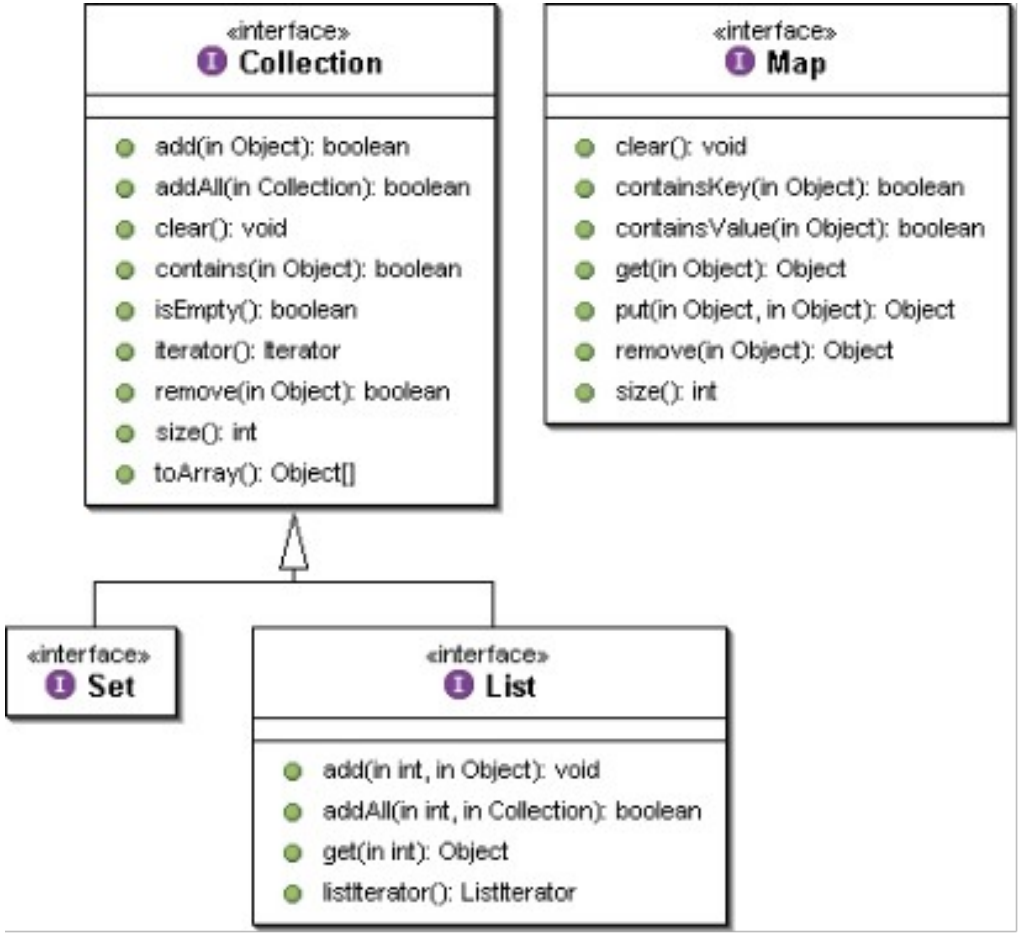
```
Set<String> cargos = new HashSet<>();

cargos.add("Gerente");
cargos.add("Diretor");
cargos.add("Presidente");
cargos.add("Secretária");
cargos.add("Funcionário");
cargos.add("Diretor");

System.out.println(cargos);
```

Set

- **Não há manipulação de índices**
- **Vantagem: performance superior em pesquisa**
  - Método contains, por exemplo

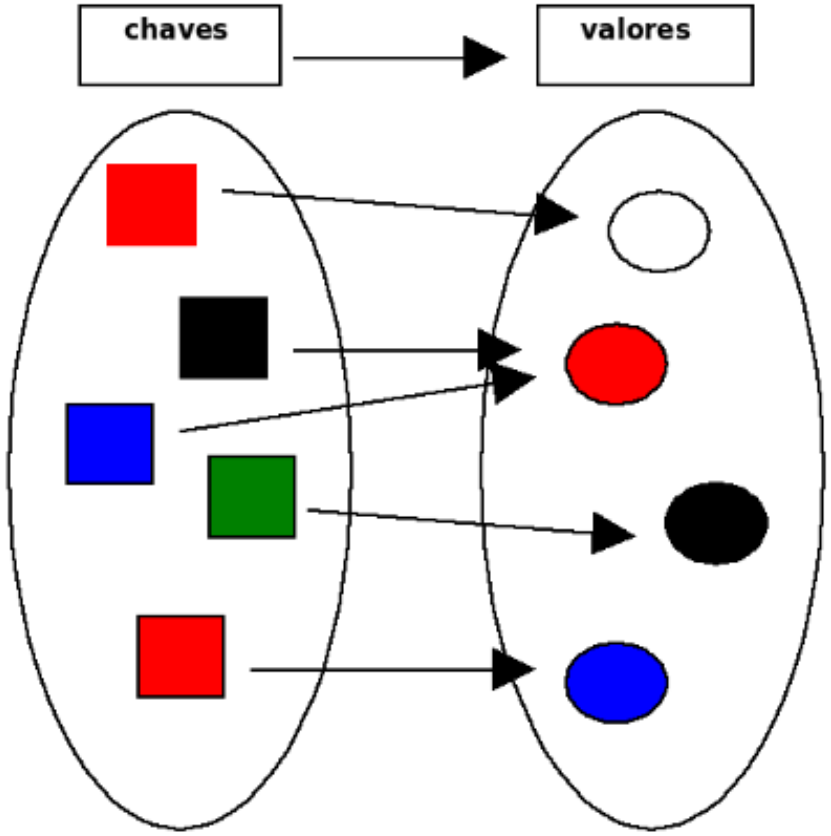
- **TreeSet**
  - Permite inserção ordenada, para que seja possível percorrer em ordem (interface Comparable)
- **LinkedHashSet**
  - Mantém a ordem de inserção dos elementos

Relembrando



Map (java.util.Map)

- **Não implementa Collection**
- **Faz o mapeamento entre chaves e valores**



Possíveis ações em um mapa:

Mapeie uma chave a um valor
O que está mapeado na chave X?
Remapeie uma certa chave
Quero o conjunto de chaves.
Quero o conjunto de valores.
Desmapeie a chave X.

Map

- **Métodos mais usados**
  - put(Object, Object): recebe a chave e o valor de uma nova associação.
  - get(Object): para saber o que está associado a um determinado objeto-chave

```
ContaCorrente c1 = new ContaCorrente();
c1.deposita(10000);
ContaCorrente c2 = new ContaCorrente();
c2.deposita(3000);

Map<String, ContaCorrente> mapaDeContas = new HashMap<>();

mapaDeContas.put("diretor", c1);
mapaDeContas.put("gerente", c2);

ContaCorrente contaDoDiretor = mapaDeContas.get("diretor");
System.out.println(contaDoDiretor.getSaldo());
```

Map

- **Algumas implementações**
  - HashMap: não sincronizado
  - TreeMap: ordenado, não sincronizado
  - HashTable: sincronizado, não permite chaves ou valores nulos
  - Properties: mapeia strings

- **O método keySet() retorna um Set com as chaves do mapa**

- **Método values() retorna Collection com todos os valores que foram associados a alguma das chaves**

Percorrendo Coleções

- **Se for uma lista, podemos utilizar um laço for, invocando o método get para cada elemento**
- **E se a coleção não permitir indexação?**
  - Exemplo: Set
- **Enhanced-for ("foreach")**

```
Set<String> conjunto = new HashSet<>();
conjunto.add("java");
conjunto.add("vraptor");
conjunto.add("scala");

for (String palavra : conjunto) {
   System.out.println(palavra);
}
```

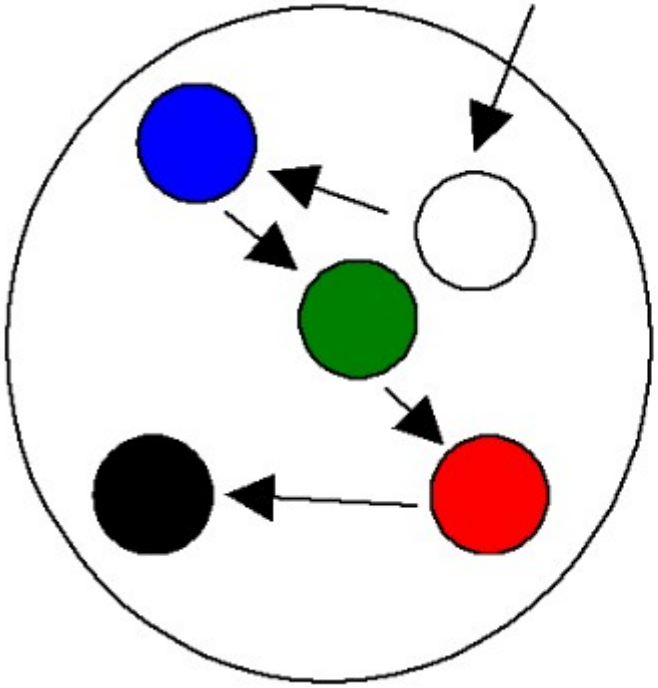
Percorrendo Coleções

- **Ordem de acesso dos elementos**
  - Lista: os elementos aparecerão de acordo com o índice em que foram inseridos
  - Conjunto: depende da implementação da interface Set

- **Vantagens de Set**
  - Verificação de existência geralmente mais rápida que outras coleções, como List
  - Se não houver a necessidade de guardar os elementos em determinada ordem e buscá-los através de um índice, o Set deve ser considerado

Percorrendo Coleções - Iterator

- **Toda coleção fornece acesso a um iterator**
- **Objeto que conhece internamente a coleção e dá acesso a seus elementos**



Possíveis ações em um iterador:

- Existe um próximo elemento na coleção?.
- Pegue o próximo elemento.
- Remova o elemento atual da coleção.

```
ContaCorrente c =
  (ContaCorrente) iterator.next();
```

Percorrendo Coleções - Iterator

- **Apesar de o mais comum atualmente ser usar o enhanced-for, o Iterator ainda é usado.**
  - O enhanced-for é um açúcar sintático que usa iterator

- **Uso**
  - Criar um Iterator que entra na coleção (pode ser obtido através do método iterator())
  - A cada chamada do método next, ele retorna o próximo elemento do conjunto

Percorrendo Coleções - Iterator

- **Exemplo**

```
Set<String> conjunto = new HashSet<>();
conjunto.add("item 1");
conjunto.add("item 2");
conjunto.add("item 3");

// retorna o iterator
Iterator<String> i = conjunto.iterator();

while (i.hasNext()) {
   // recebe a palavra
   String palavra = i.next();
   System.out.println(palavra);
}
```

Percorrendo Coleções - Iterator

- **ListIterator**
  - Recursos específicos para listas (e.g. voltar para o elemento que foi iterado anteriormente)

Equals e Hashcode

- **Muitas das coleções do java guardam os objetos dentro de tabelas de hash**
- **Cada objeto é "classificado" pelo seu hashCode**
  - Assim é possível espalhar cada objeto, agrupando-os pelo hashCode
- **Quando buscamos determinado objeto, só vamos procurar entre os elementos que estão no grupo daquele hashCode**
  - Dentro desse grupo, o objeto procurado é comparado ao candidato usando equals()

Equals e Hashcode

- **Para que isso funcione:**

  a.equals(b) implica a.hashCode() == b.hashCode()

- **Implementar hashCode sem seguir esse contrato faz com que ele retorne valores diferentes para dois objetos considerados equals**
  - Resultará em collections que usam espalhamento mas não acham objetos iguais dentro de uma mesma coleção
  - Usará o hashcode() default da classe Object
    - Baseado na transformação do endereço do objeto alocado

Equals e Hashcode

- **Objetos onde equals retorna true terão hashCodes provavelmente diferentes**
  - Possíveis diferentes hash buckets
  - Contains procura por elementos no bucket que corresponde ao hashcode do elemento procurado

- **Sempre que o método equals() for sobrecarregado para uma classe, é método hashCode() também deve ser**

- **Importante**
  - Usar @Override quando for definir estes métodos
  - No método equals(), verificar e fazer o cast do parâmetro
  - hashCode deve depender principalmente dos campos comparados em equals

Equals e Hashcode

```
public class Point {
   private final int x; private final int y;

   public Point(int x, int y) { this.x = x; this.y = y; }
   public int getX() { return x; }
   public int getY() { return y; }

   @Override
   public boolean equals(Object other) {
      boolean result = false;
      if (other instanceof Point) {
         Point that = (Point) other;
         result = (this.getX() == that.getX() && this.getY() == that.getY());
      }
      return result;
   }

   @Override
   public int hashCode() {
      return (41 * (41 + getX()) + getY());
   }
}
```

Boas Práticas

- **Grande parte das coleções usam, internamente, um array para armazenar os seus dados**
- **Quando esse array não é mais suficiente, é criado um maior e o conteúdo da antigo é copiado**
- **Este processo pode acontecer muitas vezes**
  - Coleções que crescem muito
- **Deve-se criar uma coleção já com uma capacidade grande**
  - Evita o excesso de redimensionamento

Boas Práticas

- **Evite usar coleções que guardam os elementos pela sua ordem de comparação quando não há necessidade**
  - TreeSet gasta O(log(n)) para inserir
  - HashSet gasta O(1)

Boas Práticas

- **Não se deve iterar sobre uma List utilizando um for de 0 até list.size() e usando get(int) para receber os objetos**
- **Algumas implementações da List não são de acesso aleatório como a LinkedList, fazendo esse código ter uma baixa performance computacional**
  - Nesse caso, deve-se usar o Iterator

Referências

- **Caelum apostilas. Collections framework**
  - http://www.caelum.com.br/apostila-java-orientacao-objetos/collections-framework/
- **Documentação do Java**
  - http://docs.oracle.com/javase/tutorial/collections/

- **Leitura recomendada:**
  - Martin Odersky, Lex Spoon, and Bill Venners. How to Write an Equality Method in Java. 2009.
    - http://www.artima.com/lejava/articles/equality.html